Notebook 8 [Assessment]: RAG Evaluation

Welcome to the last notebook of the course! In the previous notebook, you integrated a vector store solution into a RAG pipeline! In this notebook, you will take that same pipeline and evaluate it using numerical RAG evaluation techniques incorporating LLM-as-a-Judge metrics!

Learning Objectives:

- Learn how to integrate the techniques from prior notebooks to numerically approximate the goodness of your RAG pipeline.
- **Final Exercice**: ***By working through this notebook in the Course Environment,* you will be able to submit the coding component of the course!**

Questions To Think About:

- As you go along, remember what our metrics actually represent. Should our pipeline pass these objectives? Is our judge LLM sufficient for evaluating the pipeline? Does a particular metric even matter for our use case?
- If we left the vectorstore-as-a-memory component in our chain, do you think it would still pass the evaluation? Additionally, is the evaluation useful for assessing vectorstore-as-a-memory performance?

Environment Setup:

```
# %pip install -q langchain langchain-nvidia-ai-endpoints gradio rich
# %pip install -q arxiv pymupdf faiss-cpu ragas

## If you encounter a typing-extensions issue, restart your runtime
and try again
# from langchain_nvidia_ai_endpoints import ChatNVIDIA
# ChatNVIDIA.get_available_models()

from functools import partial
from rich.console import Console
from rich.style import Style
from rich.theme import Theme

console = Console()
base_style = Style(color="#76B900", bold=True)
```

```
norm_style = Style(bold=True)
pprint = partial(console.print, style=base_style)
pprint2 = partial(console.print, style=norm_style)

from langchain_nvidia_ai_endpoints import ChatNVIDIA, NVIDIAEmbeddings

# NVIDIAEmbeddings.get_available_models()
embedder = NVIDIAEmbeddings(model="nvidia/llama-3.2-nv-embedqa-1b-v2",
truncate="END")

#
ChatNVIDIA.get_available_models(base_url="http://llm_client:9000/v1")
instruct_llm = ChatNVIDIA(model="meta/llama-3.1-8b-instruct")
```

Part 1: Pre-Release Evaluation

In our previous notebook, we successfully combined several concepts to create a document chatbot with the aim of responsive and informative interactions. However, the diversity of user interactions necessitates comprehensive testing to truly understand the chatbot's performance. Thorough testing in varied scenarios is crucial to ensure that the system is not only robust and versatile but also aligns with user and provider expectations.

After defining your chatbot's roles and implementing the necessary features, evaluating it becomes a multi-stage process:

- **Typical Use Inspection:** Start by testing scenarios most relevant to your use case. See if your chatbot can reliably navigate discussions with limited human intervention.
  - Additionally, identify limitations or compartments that should be redirected to a human for inspection/supervision (i.e., human swap-in to confirm transactions or perform sensitive navigation) and implement those options.
- **Edge Case Inspection:** Explore the boundaries of typical use, identifying how the chatbot handles less common but plausible scenarios.
  - Before any public release, assess critical boundary conditions that could pose liability risks, such as the potential generation of inappropriate content.
  - Implement well-tested guardrails on all outputs (and possibly inputs) to limit undesired interactions and redirect users into predictable conversation flows.
- **Progressive Rollout:** Rolling out your model to a limited audience (first internal, then A/B) and implement analytics features like usage analytics dashboards and feedback avenues (flag/like/dislike/etc).

Of these three steps, the first two can be done by a small team or an individual and should be iterated on as part of the development process. Unfortunately, this needs to be done frequently and can be prone to human error. **Luckily for us, LLMs can be used to help out with LLM-as-a-Judge formulations!**

(Yeah, this probably isn't surprising by now. LLMs being strong is why this course is here...).

Part 2: LLM-as-a-Judge Formulation

In the realm of conversational AI, using LLMs as evaluators or 'judges' has emerged as a useful approach for configurable automatic testing of natural language task performance:

- An LLM can simulate a range of interaction scenarios and generate synthetic data, allowing an evaluation developer to generate targeted inputs to eliciting a range of behaviors from your chatbot.
- The chatbot's correspondence/retrieval on the synthetic data can be evaluated or parsed by an LLM and a consistent output format such as "Pass"/"Fail", similarity, or extraction can be enforced.
- Many such results can be aggregated and a metric can be derived which explains something like "% of passing evaluations", "average number of relevant details from the sources", "average cosine similarity", etc.

This idea of using LLMs to test out and quantify chatbot quality, known as **"LLM-as-a-Judge,"** allows for easy test specifications that align closely with human judgment and can be fine-tuned and replicated at scale.

There are several popular frameworks for off-the-shelf judge formulations including:

- **RAGAs (short for RAG Assessment)**, which offers a suite of great starting points for your own evaluation efforts.
- **LangChain Evaluators**, which are similar first-party options with many implicitly-constructible agents.

Instead of using the chains as-is, we will instead expand on the ideas and evaluate our system with a more custom solution.

Part 3: [Assessment Prep] Pairwise Evaluator

The following exercise will flesh out a custom implementation of a simplified LangChain Pairwise String Evaluator.

To prepare for our RAG chain evaluation, we will need to:

- Pull in our document index (the one we saved in the previous notebook).
- Recreate our RAG pipeline of choice.

We will specifically be implementing a judge formulation with the following steps:

- Sample the RAG agent document pool to find two document chunks.
- Use those two document chunks to generate a synthetic "baseline" question-answer pair.
- Use the RAG agent to generate its own answer.
- Use a judge LLM to compare the two responses while grounding the synthetic generation as "ground-truth correct."

The chain should be a simple but powerful process that tests for the following objective:

> ***Does my RAG chain outperform a narrow chatbot with limited document access.***

This will be the system used for the final evaluation! To see how this system is integrated into the autograder, please check out the implementation in `frontend/server_app.py`.

Task 1: Pull In Your Document Retrieval Index

For this exercise, you will pull in the `docstore_index` file you created as part of your earlier notebook. The following cell should be able to load in the store as-is.

```
## Make sure you have docstore_index.tgz in your working directory
from langchain_nvidia_ai_endpoints import NVIDIAEmbeddings
from langchain_community.vectorstores import FAISS

# embedder = NVIDIAEmbeddings(model="nvidia/embed-qa-4",
truncate="END")

!tar xzvf docstore_index.tgz
docstore = FAISS.load_local("docstore_index", embedder,
allow_dangerous_deserialization=True)
docs = list(docstore.docstore._dict.values())

def format_chunk(doc):
    return (
        f"Paper: {doc.metadata.get('Title', 'unknown')}"
        f"\n\nSummary: {doc.metadata.get('Summary', 'unknown')}"
        f"\n\nPage Body: {doc.page_content}"
    )

## This printout just confirms that your store has been retrieved
pprint(f"Constructed aggregate docstore with
{len(docstore.docstore._dict)} chunks")
```

```
pprint(f"Sample Chunk:")
print(format_chunk(docs[len(docs)//2]))
```

Task 2: [Exercise] Pull In Your RAG Chain

Now that we have our index, we can recreate the RAG agent from the previous notebook!

Key Modifications:

- To keep things simple, feel free to disregard the vectorstore-as-a-memory component. Incorporating it will require some more overhead and will make the exercise a bit more complicated.

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableLambda, RunnableBranch
from langchain_core.runnables.passthrough import RunnableAssign
from langchain.document_transformers import LongContextReorder

from langchain_nvidia_ai_endpoints import ChatNVIDIA, NVIDIAEmbeddings

from functools import partial
from operator import itemgetter

import gradio as gr

#####################################################################

# NVIDIAEmbeddings.get_available_models()
embedder = NVIDIAEmbeddings(model="nvidia/llama-3.2-nv-embedqa-1b-v2",
truncate="END")

# ChatNVIDIA.get_available_models()
instruct_llm = ChatNVIDIA(model="meta/llama-3.1-8b-instruct")
llm = instruct_llm | StrOutputParser()

#####################################################################

def docs2str(docs, title="Document"):
    """Useful utility for making chunks into context string. Optional,
but useful"""
    out_str = ""
    for doc in docs:
        doc_name = getattr(doc, 'metadata', {}).get('Title', title)
        if doc_name: out_str += f"[Quote from {doc_name}] "
        out_str += getattr(doc, 'page_content', str(doc)) + "\n"
    return out_str

chat_prompt = ChatPromptTemplate.from_template(
```

```
    "You are a document chatbot. Help the user as they ask questions
about documents."
    " User messaged just asked you a question: {input}\n\n"
    " The following information may be useful for your response: "
    " Document Retrieval:\n{context}\n\n"
    " (Answer only from retrieval. Only cite sources that are used.
Make your response conversational)"
    "\n\nUser Question: {input}"
)

def output_puller(inputs):
    """Output generator. Useful if your chain returns a dictionary
with key 'output'"""
    if isinstance(inputs, dict):
        inputs = [inputs]
    for token in inputs:
        if token.get('output'):
            yield token.get('output')

######################################################################
## TODO: Pull in your desired RAG Chain. Memory not necessary

## Chain 1 Specs: "Hello World" -> retrieval_chain
##   -> {'input': <str>, 'context' : <str>}
long_reorder =
RunnableLambda(LongContextReorder().transform_documents)  ## GIVEN
context_getter = RunnableLambda(lambda x: x)  ## TODO
retrieval_chain = {'input' : (lambda x: x)} |
RunnableAssign({'context' : context_getter})

## Chain 2 Specs: retrieval_chain -> generator_chain
##   -> {"output" : <str>, ...} -> output_puller
generator_chain = RunnableLambda(lambda x: x)  ## TODO
generator_chain = {'output' : generator_chain} |
RunnableLambda(output_puller)  ## GIVEN

## END TODO
######################################################################

rag_chain = retrieval_chain | generator_chain

# pprint(rag_chain.invoke("Tell me something interesting!"))
for token in rag_chain.stream("Tell me something interesting!"):
    print(token, end="")
```

Step 3: Generating Synthetic Question-Answer Pairs

In this section, we can implement the first few part of our evaluation routine:

- **Sample the RAG agent document pool to find two document chunks.**
- **Use those two document chunks to generate a synthetic "baseline" question-answer pair.**
- Use the RAG agent to generate its own answer.
- Use a judge LLM to compare the two responses while grounding the synthetic generation as "ground-truth correct."

The chain should be a simple but powerful process that tests for the following objective:

> Does my RAG chain outperform a narrow chatbot with limited document access?

```
import random

num_questions = 3
synth_questions = []
synth_answers = []

simple_prompt = ChatPromptTemplate.from_messages([('system',
'{system}'), ('user', 'INPUT: {input}')])

for i in range(num_questions):
    doc1, doc2 = random.sample(docs, 2)
    sys_msg = (
        "Use the documents provided by the user to generate an
interesting question-answer pair."
        " Try to use both documents if possible, and rely more on the
document bodies than the summary."
        " Use the format:\nQuestion: (good question, 1-3 sentences,
detailed)\n\nAnswer: (answer derived from the documents)"
        " DO NOT SAY: \"Here is an interesting question pair\" or
similar. FOLLOW FORMAT!"
    )
    usr_msg = (
        f"Document1: {format_chunk(doc1)}\n\n"
        f"Document2: {format_chunk(doc2)}"
    )

    qa_pair = (simple_prompt | llm).invoke({'system': sys_msg,
'input': usr_msg})
    synth_questions += [qa_pair.split('\n\n')[0]]
    synth_answers += [qa_pair.split('\n\n')[1]]
    pprint2(f"QA Pair {i+1}")
    pprint2(synth_questions[-1])
    pprint(synth_answers[-1])
    print()
```

Step 4: Answer The Synthetic Questions

In this section, we can implement the third part of our evaluation routine:

- Sample the RAG agent document pool to find two document chunks.
- Use those two document chunks to generate a synthetic "baseline" question-answer pair.
- **Use the RAG agent to generate its own answer.**
- Use a judge LLM to compare the two responses while grounding the synthetic generation as "ground-truth correct."

The chain should be a simple but powerful process that tests for the following objective:

> Does my RAG chain outperform a narrow chatbot with limited document access?

```
## TODO: Generate some synthetic answers to the questions above.
##   Try to use the same syntax as the cell above
rag_answers = []
for i, q in enumerate(synth_questions):
    ## TODO: Compute the RAG Answer
    rag_answer = ""
    rag_answers += [rag_answer]
    pprint2(f"QA Pair {i+1}", q, "", sep="\n")
    pprint(f"RAG Answer: {rag_answer}", "", sep='\n')
```

Step 5: Implement A Human Preference Metric

In this section, we can implement the fourth part of our evaluation routine:

- Sample the RAG agent document pool to find two document chunks.
- Use those two document chunks to generate a synthetic "baseline" question-answer pair.
- Use the RAG agent to generate its own answer.
- **Use a judge LLM to compare the two responses while grounding the synthetic generation as "ground-truth correct."**

The chain should be a simple but powerful process that tests for the following objective:

> Does my RAG chain outperform a narrow chatbot with limited document access?

```
## TODO: Adapt this prompt for whichever LLM you're actually
interested in using.
## If it's llama, maybe system message would be good?
eval_prompt = ChatPromptTemplate.from_template("""INSTRUCTION:
Evaluate the following Question-Answer pair for human preference and
consistency.
Assume the first answer is a ground truth answer and has to be
correct.
```

```
Assume the second answer may or may not be true.
[1] The second answer lies, does not answer the question, or is
inferior to the first answer.
[2] The second answer is better than the first and does not introduce
any inconsistencies.

Output Format:
[Score] Justification

{qa_trio}

EVALUATION:
""")

pref_score = []

trio_gen = zip(synth_questions, synth_answers, rag_answers)
for i, (q, a_synth, a_rag) in enumerate(trio_gen):
    pprint2(f"Set {i+1}\n\nQuestion: {q}\n\n")

    qa_trio = f"Question: {q}\n\nAnswer 1 (Ground Truth): {a_synth}\n\
n Answer 2 (New Answer): {a_rag}"
    pref_score += [(eval_prompt | llm).invoke({'qa_trio': qa_trio})]
    pprint(f"Synth Answer: {a_synth}\n\n")
    pprint(f"RAG Answer: {a_rag}\n\n")
    pprint2(f"Synth Evaluation: {pref_score[-1]}\n\n")
```

Congratulations! We now have an LLM system that reasons about our pipeline and tries to evaluate it! Now that we have some judge results, we can simply aggregate the results and see how often our formulation was according to an LLM:

```
pref_score = sum(("[2]" in score) for score in pref_score) /
len(pref_score)
print(f"Preference Score: {pref_score}")
```

Part 4: Advanced Formulations

The exercise above was meant to prepare you for the final assessment of the course and showcased a simple but effective evaluator chain. The objective and implementation details were provided for you, and the logic for using it probably makes sense now that you've seen it in action.

With that being said, this metric was merely a product of us specifying:

- **What kind of behavior is important for our pipeline to have?**
- **What do we need to do in order to exhibit and evaluate this behavior?**

From these two questions, we could have come up with plenty of other evaluation metrics that could have assessed different attributes, incorporated different evaluator chain techniques, and even required different pipeline organization strategies. Though far from an exhaustive list, some common formulations you will likely come across may include:

- **Style Evaluation:** Some evaluation formulations can be as simple as "let me ask some questions and see if the output feels desirable." This might be used to see whether a chatbot "acts like it's supposed to" based on a description provided to a judge LLM. We're using quotations since this kind of assessment can reasonably be achieved with nothing but prompt engineering and a while loop.

- **Ground-Truth Evaluation:** In our chain, we used synthetic generation to create some random questions and answers using a sampling strategy, but in reality you may actually have some representative questions and answers that you need your chatbot to consistently get right! In this case, a modification of the exercise chain above should be implemented and closely monitored as you develop your pipelines.

- **Retrieval/Augmentation Evaluation:** This course made many assumptions about what kinds of preprocessing and prompting steps would be good for your pipelines, and much of this was determined by experimentation. Factors such as document preprocessing, chunking strategies, model selection, and prompt specification all played important roles, so creating metrics to validate these decisions may be of interest. This kind of metric might require your pipeline to output your context chunks or may even rely solely on embedding similarity comparisons, so keep this in mind when trying to implement a chain that works with multiple evaluation strategies. Consider the **RagasEvaluatorChain** abstraction as a decent starting point for making an custom generalizable evaluation routine.

- **Trajectory Evaluation:** Using more advanced agent formulations, you can implement multiple-query strategies that assume the presence of conversational memory. With this, you can implement an evaluation agent which can:

  - Ask a series of questions in order to evaluate how well the agent is able to adapt and cater to the scenario. This kind of system generally considers a series of correspondence and aims to tease out and evaluate a "trajectory" of how the agent navigated the conversation. The **LangChain Trajectory Evaluators documentation** is a good starting point.
  - Alternatively, you could also implement an evaluation agent that tries to achieve objectives by interacting with the chatbot. Such an agent can output whether they were able to navigate to their solution in a natural manner, and can even be used to generate a report about the percieved performance. The **LangChain Agents documentation** is a good starting point!

At the end of the day, just make sure to use the tools you have at your disposal appropriately. By this point in the course, you should already be well-acquainted with the LLM core value propositions: **They're powerful, scalable, predictable, controllable, and orchestratable... but will act unpredictably when you just expect them to work by default.** Assess your needs, formulate and validate your pipelines, give enough information, and add as much control as you can to make your system work consistently, efficiently, and effectively.

Part 5: [Assessment] Evaluating For Credit

Welcome to the last exercise of the course! Hopefully you've enjoyed the material and are ready to actually get credit for these notebooks! For this part:

- **Make sure you're in the course environment**
- **Make sure `docstore_index/` has been uploaded to the course environment...**
  - **...and contains at least one Arxiv paper which has been updated recently.**
- **Make sure you don't have some old session of `09_langserve.ipynb` already occupying the port. Your assessment requires you to implement the new `/retriever` and `/generator` endpoints!!**

Objective: On launch, **`frontend/frontend_block.py`** had several lines of code which trigger the course pass condition. Your objective is to invoke that series of commands by using your pipeline to pass the **Evaluation** check! Recall `09_langserve.ipynb` and use it as a starting example! As a recommendation, consider duplicating it so that you can keep the original as an authoritative reference.

Once Finished: While your course environment is still open, please navigate back to your course environment launcher area and click the **"Assess Task"** button! After that, you're all done!

```
%%js
var url = 'http://'+window.location.host+':8090';
element.innerHTML = '<a style="color:green;" target="_blank"
href='+url+'><h1>< Link To Gradio Frontend ></h1></a>';
```

Congratulations On Completing The Course

Hopefully this course was not only exciting and challenging, but also adequately prepared you for work on the cutting edge of LLM and RAG system development! Going forward, you should have the skills necessary to tackle industry-level challenges and explore RAG deployment with open-source models and frameworks.

Some NVIDIA-specific releases related to this that you may find interesting include:

- **NVIDIA NIM**, which offers microservice spinup routines that can be deployed on local compute.
- **TensorRT-LLM** is the current recommended framework for deploying GPU-accelerated LLM model engines in production settings.
- **NVIDIA's Generative AI Examples Repo**, which includes the current canonical microservice example application and will be updated with new resources as new production workflows get released.
- **The Knowledge-Based Chatbot Technical Brief** which discusses additional publicly-accessible details on productionalizing RAG systems.

Additionally, some key topics you may be interested in delving more into include:

- **LlamaIndex**, which has strong components that can augment and occasionally improve upon the LangChain RAG features.
- **LangSmith**, an upcoming agent productionalization service offered by LangChain.
- **Gradio**, though touched on in the course, has many more interface options which will be worth investigating. For inspiration, consider checking out **HuggingFace Spaces** for examples.
- **LangGraph** is a framework for graph-based LLM orchestration, and is a natural next step forward for those interested in multi-agent workflows.
- **DSPy**, a flow engineering framework that allows you to optimize LLM orchestration pipelines based on empirical performance results.